Veri Yapıları

Ortam Kurulumu, Ayarı ve Uygulamalar

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

İçerik-Uygulama

- Geliştirme Ortamının Kurulumu (IDE, Compiler)
- Temel C++ Programı Yazma ve Derleme
- Sınıf (Class) Tanımlama ve Nesne (Object) Oluşturma
- İşaretçi (Pointer) ve Referans (Reference) Kullanımı
- Basit Bir Veri Yönetim Sınıfı Tasarımı

Ortam Kurulum

- IDE Kurulumu:
  - JetBrains Toolbox ve CLion Kurulumu
  - Lisanslama
    - Lab Lisansı
    - Kişisel ücretsiz edu.tr lisansı
- Compiler Ayarı
  - Yol 1: CLion ile gelen bundle yolunu ortam değişkenlerinde PATH'e ekleme
  - Yol 2: MinGW gibi derleyicileri sıfırdan kurup, ortam değişkenine ekleyip, son olarak da IDE'ye ekleme

Yol 2: Sıfırdan MinGW Kurulumu ve Clion Entegrasyonu

- https://www.msys2.org/ adresinden msys indirilip kurulur
- Kurulum yolu genellikle C:\msys64
- Kurulum sonrası:
  - MSYS2 UCRT64 terminali açılır
  - “pacman –Syu” komutu çalıştırılır
    - Bu noktada terminal kapanabilir, hata verebilir, başarılı olana kadar tekrarlanmalı
    - Güncellemelerin hepsinin tamamlandığından emin olun
  - “pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain” komutu çalıştırılır
    - Paketlerin hangilerinin kurulacağı sorulacak Enter tuşuna basarak hepsinin kurulması gerektiğini söylemiş olun
    - Kuruluma devam etmek için sorulan sorulara Y ile cevap verin (ENTER)

Yol 2: Sıfırdan MinGW Kurulumu ve Clion Entegrasyonu

- Kurulumlar bittikten sonra:
  - Msys2 dizini C:\msys64 üzerinde kurulu olan MinGW dizini olan C:\msys64\ucrt64\bin yolunu ortam değişkenlerinde PATH'e ekleyin
  - Terminal'I açıp aşağıdaki komutlar ile versiyonu doğrulayın
    - "gcc --version"
    - "g++ --version"
    - "gdb --version"

Yol 2: Sıfırdan MinGW Kurulumu ve Clion Entegrasyonu

- CLion Ayarları
  - File>Settings menüsünde “Build, Execution, Deployment > Toolchains” sekmesine tıklayın
  - MinGW’yi silebilirsiniz yada yeni birtane ekleyin
  - Otomatik versiyonları bulmadığında manuel ayarlama yapın
    - C:\msys64\ucrt64 yolu verilir
    - Tüm derleyici dosyaları yukarıdaki yoldaki bin klasörü içindedir
  - Ana derleyici olmasını istiyorsanız MinGW’yi soldan en üst sıraya alın

Proje Yapısı

- cmake-build klasörü
- CMakeLists.txt
  - Tüm derleme ayarları
  - cmake_minimum_required, project, set, add_executable ...
  - Alt uygulamalar için birden fazla tanımlama >> add_executable

Bazı Temel ve İleri İşlemler

- Run
- Run Configuration (Run > Edit Configurations)
- Build
- Clean Build
- Rebuild
- Reload Cmake Project
- File > Invalidate Caches
  - “Clear file system cache and Local History”
- Tools > Cmake > ...

İlk Program

- İlk Program: "Merhaba, Veri Yapıları!“
  - iostream kütüphanesinin kullanımı.
  - main fonksiyonu.
  - cin ile girdi alma ve cout ile ekrana yazı yazdırma.
  - Programı derleme (compile) ve çalıştırma (run) adımları.

İlk Program

```cpp
#include <iostream>

int main() {
    int x;
    std::cout << "Lutfen bir sayi giriniz: ";
    std::cin >> x;
    std::cout << "Girdiginiz Sayi: " << x << std::endl;
}
```

Çıktı

```
Lutfen bir sayi giriniz: 9
Girdiginiz Sayi: 9
```

Uygulama 1: Dikdortgen Sınıf Tasarımı

1. Dikdortgen.h dosyası oluşturun:
   a. private üyeler: genislik, yukseklik (double tipinde).
   b. public üyeler:
      i. Bir varsayılan yapıcı metot (constructor).
      ii. Genişlik ve yüksekliği parametre alan bir yapıcı metot.
      iii. alanHesapla() metodu.
      iv. cevreHesapla() metodu.
      v. bilgileriYazdir() metodu.
2. Dikdortgen.cpp dosyası oluşturun:
   a. Metotların gövdelerini (implementasyonlarını) yazın.
3. main.cpp dosyası oluşturun:
   a. Dikdortgen sınıfından birkaç nesne (örneğin, d1 ve d2) oluşturun.
   b. Nesnelerin metotlarını çağırarak alan ve çevrelerini hesaplatıp ekrana yazdırın.

Uygulama 1: Dikdortgen Sınıf Tasarımı

```
1   #ifndef DIKDORTGEN_H
2   #define DIKDORTGEN_H
3
4   class Dikdortgen {
5       private:
6           double genislik;
7           double yukseklik;
8
9       public:
10          Dikdortgen();
11          Dikdortgen(double g, double y);
12          double alanHesapla();
13          double cevreHesapla();
14          void bilgileriYazdir();
15  };
16
17
18  #endif #ifndef DIKDORTGEN_H
```

Dikdortgen.h

Uygulama 1: Dikdortgen Sınıf Tasarımı

```cpp
#include <iostream>
#include "Dikdortgen.h"

Dikdortgen::Dikdortgen() {
    genislik = 0.0;
    yukseklik = 0.0;
}

Dikdortgen::Dikdortgen(double g, double y) {
    genislik = g;
    yukseklik = y;
}

double Dikdortgen::alanHesapla() {
    return genislik * yukseklik;
}

double Dikdortgen::cevreHesapla() {
    return 2 * (genislik + yukseklik);
}

void Dikdortgen::bilgileriYazdir() {
    std::cout << "------------------------" << std::endl;
    std::cout << "Genislik: " << genislik << std::endl;
    std::cout << "Yukseklik: " << yukseklik << std::endl;
    std::cout << "Alan: " << alanHesapla() << std::endl;
    std::cout << "Cevre: " << cevreHesapla() << std::endl;
    std::cout << "------------------------" << std::endl;
}
```

Dikdortgen.cpp

Uygulama 1: Dikdortgen Sınıf Tasarımı

```cpp
#include <iostream>
#include "Dikdortgen.h"

int main() {
    std::cout << "d1 nesnesi (varsayilan):" << std::endl;
    Dikdortgen d1;
    d1.bilgileriYazdir();

    std::cout << "\nd2 nesnesi (parametrelerle):" << std::endl;
    Dikdortgen d2( g: 10.5,  y: 4.0);
    d2.bilgileriYazdir();

    std::cout << "\nd2 nesnesinin alani ayrica hesaplandi: " << d2.alanHesapla() << std::endl;
    std::cout << "d2 nesnesinin cevresi ayrica hesaplandi: " << d2.cevreHesapla() << std::endl;
    return 0;
}
```

dikdortgenApp.cpp veya **main.cpp**

Uygulama 1: Dikdortgen Sınıf Tasarımı

Çıktı

```
d1 nesnesi (varsayilan):
------------------------
Genislik: 0
Yukseklik: 0
Alan: 0
Cevre: 0
------------------------

d2 nesnesi (parametrelerle):
------------------------
Genislik: 10.5
Yukseklik: 4
Alan: 42
Cevre: 29
------------------------

d2 nesnesinin alani ayrica hesaplandi: 42
d2 nesnesinin cevresi ayrica hesaplandi: 29
```

Uygulama 2: Değişken Değerlerini Değiştirme (Swap)

1. Bir main fonksiyonu içinde iki int değişken tanımlayın (a = 10, b = 20).
2. swapByValue(int x, int y) fonksiyonunu yazın ve çağırın.
   - Sonucu gözlemleyin (a ve b değişmeyecektir).
3. swapByReference(int &x, int &y) fonksiyonunu yazın ve çağırın.
   - Sonucu gözlemleyin (a ve b yer değiştirecektir).
4. swapByPointer(int *x, int *y) fonksiyonunu yazın ve çağırın.
   - Sonucu gözlemleyin (a ve b yer değiştirecektir).

Uygulama 2: Değişken Değerlerini Değiştirme (Swap)

```
void swapByValue(int x, int y) {
    int temp = x; x = y; y = temp;
    std::cout << "  swapByValue fonksiyonu icinde: x = " << x << ", y = " << y << std::endl;
}

void swapByReference(int &x, int &y) {
    int temp = x; x = y; y = temp;
}

void swapByPointer(int *x, int *y) {
    int temp = *x; *x = *y; *y = temp;
}
```

Uygulama 2: Değişken Değerlerini Değiştirme (Swap)

```cpp
#include <iostream>

void swapByValue(int x, int y){...}
void swapByReference(int &x, int &y){...}
void swapByPointer(int *x, int *y){...}

int main() {
    int a = 10; int b = 20;
    std::cout << "--- Baslangic Degerleri ---" << std::endl;
    std::cout << "a = " << a << ", b = " << b << std::endl;
    std::cout << "----------------------------" << std::endl << std::endl;

    std::cout << "1. swapByValue(a, b) cagriyor..." << std::endl;
    swapByValue( x: a,  y: b);
    std::cout << "  Sonuc: a ve b degismeyecektir." << std::endl;
    std::cout << "  a = " << a << ", b = " << b << std::endl;
    std::cout << "----------------------------" << std::endl << std::endl;

    a = 10; b = 20;
    std::cout << "Degiskenler sifirlandi: a = " << a << ", b = " << b << std::endl << std::endl;
    std::cout << "2. swapByReference(a, b) cagriyor..." << std::endl;
    swapByReference( x: [&] a,  y: [&] b);
    std::cout << "  Sonuc: a ve b yer degistirecektir." << std::endl;
    std::cout << "  a = " << a << ", b = " << b << std::endl;
    std::cout << "----------------------------" << std::endl << std::endl;

    a = 10; b = 20;
    std::cout << "Degiskenler sifirlandi: a = " << a << ", b = " << b << std::endl << std::endl;
    std::cout << "3. swapByPointer(&a, &b) cagriyor..." << std::endl;
    swapByPointer( x: &a,  y: &b);
    std::cout << "  Sonuc: a ve b yer degistirecektir." << std::endl;
    std::cout << "  a = " << a << ", b = " << b << std::endl;
    std::cout << "----------------------------" << std::endl;
    return 0;
}
```

Uygulama 2: Değişken Değerlerini Değiştirme (Swap)

Çıktı

```
--- Baslangic Degerleri ---
a = 10, b = 20
-------------------------

1. swapByValue(a, b) cagriyor...
  swapByValue fonksiyonu icinde: x = 20, y = 10
  Sonuc: a ve b degismeyecektir.
  a = 10, b = 20
-------------------------

Degiskenler sifirlandi: a = 10, b = 20

2. swapByReference(a, b) cagriyor...
  Sonuc: a ve b yer degistirecektir.
  a = 20, b = 10
-------------------------

Degiskenler sifirlandi: a = 10, b = 20

3. swapByPointer(&a, &b) cagriyor...
  Sonuc: a ve b yer degistirecektir.
  a = 20, b = 10
-------------------------
```

Uygulama 3: Dinamik Bellek Kullanımı

1. Kullanıcıdan bir tamsayı (n) alın.
2. new operatörünü kullanarak n elemanlı bir int dizisi için bellekten yer ayırın ve bir işaretçide adresini tutun.
3. Bu diziyi rastgele sayılarla doldurun ve ekrana yazdırın.
4. delete[] operatörünü kullanarak ayırdığınız belleği sisteme geri iade edin. Bu adımın önemini vurgulayın (bellek sızıntısını - memory leak - önlemek).

Uygulama 3: Dinamik Bellek Kullanımı

Devamı için sonraki slayt!

```cpp
#include <iostream>
#include <random>
#include <new>

int main() {
    int n;
    std::cout << "Dizi boyutunu girin (n): ";
    std::cin >> n;

    if (n <= 0) {
        std::cout << "Lutfen pozitif bir tamsayi girin." << std::endl;
        return 1;
    }

    int* dizi = nullptr;
    try {
        dizi = new int[n];
        std::cout << n << " elemanli bir dizi icin bellekten yer ayrildi." << std::endl;
    } catch (const std::bad_alloc& e) {
        std::cerr << "Bellek ayirma basarisiz oldu: " << e.what() << std::endl;
        return 1;
    }

    std::random_device rd;
    std::mt19937 gen(sd: rd());
    std::uniform_int_distribution<int> dist(a: 0, b: 99);

    std::cout << "\nDizi elemanlari (rastgele sayilar):" << std::endl;
    for (int i = 0; i < n; ++i) {
        dizi[i] = dist([&] gen);
        std::cout << dizi[i] << " ";
    }
    std::cout << std::endl;
    std::cout << "dizi adresi: " << dizi << std::endl;
```

Uygulama 3: Dinamik Bellek Kullanımı

```
35
36      delete[] dizi;
37      std::cout << "delete sonrasi dizi adresi: " << dizi << std::endl;
38      std::cout << "delete sonrasi dizi elemanlari (rastgele sayilar):" << std::endl;
39      for (int i = 0; i < n; ++i) {
40          std::cout << dizi[i] << " ";
41      }
42      std::cout << std::endl;
43
44      dizi = nullptr;
45      std::cout << "null pointer sonrasi dizi adresi: " << dizi << std::endl;
46      std::cout << "null pointer sonrasi dizi elemanlari bu asamada cagirilirsa uygulama hata verir. 0. noktadan veri cekemez." << std::endl;
47
48      std::cout << "\033[1;31m" << "\nBilgi" << "\033[0m" << std::endl;
49      std::cout << "\033[1;31m" << "-------------------------------------------------------" << "\033[0m" << std::endl;
50      std::cout << "\033[1;33m" << "Bellek 'delete[]' operatoru ile sisteme iade edildi." << "\033[0m" << std::endl;
51      std::cout << "\033[1;33m" << "Bu adim cok onemlidir cunku ayrilan bellegin serbest birakilmamasi," << "\033[0m" << std::endl;
52      std::cout << "\033[1;33m" << "program calismaya devam ettikce kullanilabilir bellegin azalmasina neden olan" << "\033[0m" << std::endl;
53      std::cout << "\033[1;33m" << "'bellek sizintisi' (memory leak) sorununa yol acar." << "\033[0m" << std::endl;
54      std::cout << "\033[1;31m" << "-------------------------------------------------------" << "\033[0m" << std::endl;
55
56      return 0;
57  }
```

Kodun devamı!

Uygulama 3: Dinamik Bellek Kullanımı

Çıktı

```
Dizi boyutunu girin (n): 8
8 elemanli bir dizi icin bellekten yer ayrildi.

Dizi elemanlari (rastgele sayilar):
42 40 53 66 28 17 81 15
dizi adresi: 0x1ea71315f10
delete sonrasi dizi adresi: 0x1ea71315f10
delete sonrasi dizi elemanlari (rastgele sayilar):
42 40 53 66 28 17 81 15
null pointer sonrasi dizi adresi: 0
null pointer sonrasi dizi elemanlari bu asamada cagirilirsa uygulama hata verir. 0. noktadan veri cekemez.

Bilgi
-------------------------------------------------------
Bellek 'delete[]' operatoru ile sisteme iade edildi.
Bu adim cok onemlidir cunku ayrilan bellegin serbest birakilmamasi,
program calismaya devam ettikce kullanilabilir bellegin azalmasina neden olan
'bellek sizintisi' (memory leak) sorununa yol acar.
-------------------------------------------------------
```

Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN